Physics-Informed Neural Network Implementation for Parameter Extraction in TiO_2 ReRAM Devices

Aravind Krishnan Radhakrishnan Shyni

CSIP5501

M.Sc. Artificial Intelligence

P2842499

Supervisor: Dr. Aboozar Taherkani

De Montfort University, Leicester

Abstract

Resistive Random-Access memory (ReRAM) is a promising technology for the next-generation computing systems, but its widespread adoption lies on the availability of accurate device models for circuit design and simulation. Extracting physical parameter values for these models from both experimental and simulated data can be challenging due to non-linear and time-dependent nature of ReRAM dynamics. This project investigates the application of Physics-Informed Neural Networks (PINNs) for the automated extraction of the values of parameters for a specific Titanium Dioxide (TiO_2) ReRAM model proposed by Messaris et al. (2017a).

The project involved implementing the target ReRAM model's governing differential equations within a PINN framework. Synthetic data that mimics the resistive switching behaviour of the device under various voltage stimuli was generated using known physical parameters. The PINN was designed to learn the device's resistance response and extract the underlying parameters by minimizing loss function, physics-based residuals and parameter regularization.

The results demonstrate that the PINN successfully extracted the key physical parameters of the ReRAM model with high accuracy when trained on synthetic data. The model exhibited a strong fit to the generated data and effectively satisfied the physical constraints.

Keywords: Memristor, ReRAM, Parameter Extraction, Physics-Informed Neural Network (PINNs), TiO_2.

Acknowledgment

I would like to express my gratitude to my supervisor, Dr. Aboozar Taherkani, for his guidance and support throughout this project. The expertise and encouragement provided by my supervisor were instrumental in navigating the complexities of this research. I am also thankful to De Montfort University and the Computing, engineering and Media department for providing the necessary resources and academic environment that made this work possible.

Finally, special thanks are extended to my family and friends for their constant support, patience and understanding and also whose encouragement that kept me motivated.

Table of Contents

Chapter 1. Introduction 1

1.1 Background: 1

1.2 Problem Statement: 2

1.3 Aims and Objectives: 2

1.4 Project Scope: 4

1.5 Research Questions 4

1.6 Structure of the report: 4

Chapter 2: Literature Review 6

2.1 Overview of ReRAM Technology 6

2.2 Traditional Parameter Extraction Methods 6

2.3 Physics-Informed Neural Networks (PINNs) 7

2.3.1 Concept and Architecture 7

2.3.2 Incorporating Physical Laws 7

2.3.3 Applications of PINNs 8

2.3.4 Advantages and Limitations of PINNs 8

2.4 Memristor Modelling: Approaches and Challenges 8

2.4.1 Classification of Memristor Models 9

2.4.2 Review of Prominent Memristor Models 9

2.4.3 The Messaris et al. (2017) TiO_2 ReRAM Model 9

2.4.4 Challenges in Memristor Modelling and Parameter Extraction 10

2.5 Summary and Identification of Research Gap 10

Chapter 3: Methodology 12

3.1 Overview of the Research Approach 12

3.2 The TiO_2 ReRAM Switching Rate Model 12

3.3 Synthetic Data generation 13

3.4 Physics-Informed Neural network (PINN) Design and Implementation 14

3.4.1 Neural Network Architecture (`self.net`) 14

3.4.2 Input and Output Normalization 15

3.4.3 Trainable Physical Parameters 15

3.5 Composite Loss Function 15
3.5.1 Data Loss ($\mathcal{L}data$) 16
3.5.2 Physics Loss ($\mathcal{L}physics$) 16
3.5.3 Regularization ($\mathcal{L}reg$) 16
3.6 Training and Optimization 16
3.7 Evaluation metrics 17
Chapter 4: Implementation and Results 18
4.1 Experimental Setup and Configuration 18
4.1.1 Target Parameters and Initialization 18
4.1.2 Data Generation 18
4.1.3 Training Configuration 19
4.2 Training Convergence Analysis 19
4.3 Extracted Physical Parameters 19
4.4 Parameter Evolution During Training (Best Result Run) 21
4.5 Data Fitting Performance 23
4.6 Physics Residual Analysis 24
4.7 Learned Physical Functions 25
4.8 Summary of Results 26
Chapter 5: Critical Evaluation 27
5.1 Interpretation and Accuracy of Extracted Parameters 27
5.2 Analysis of Model Performance and Loss Convergence 28
5.3 Impact of Initial Parameter Values 28
5.4 Evaluation of Data Fidelity and Physics Conformance 28
5.5 Strengths of the Physics-Informed Approach 29
5.6 Limitations of the Study and Methodological Considerations 29
5.7 Implications of findings and Broader Context 30
Chapter 6: Conclusion 32
6.1 Summary of Principal Findings 32
6.2 Achievements of Aims and Objectives 32
6.3 Contributions of the Project 33
6.4 Recommendations for Future Work 33

Chapter 1. Introduction

1.1 Background:

Resistive Random Access Memory (ReRAM) is a new type of non-volatile memory that largely finds its uses in neuromorphic computing and efficient high-density storage. These were brought about by Leon Chua in 1971 as the fourth fundamental passive circuit element (Chua, 1971) and substantiated experimentally by Strukov et al. (2008) using Titanium dioxide (TiO2). The memristor can change the electrical resistance depending on the history of voltage applied. Because of this particular feature ReRAMs become the best candidate for high-density non-volatile memory applications, along with a potential part in bio-inspired neuromorphic computing systems and in-memory architectures (Serb et al., 2016; Yu, 2018). TiO2-based ReRAM, which can also be CMOS compatible, offers low power consumption and high scalability for next-generation memory and computing architectures.

The resistance-switching behaviour of ReRAM devices is ruled by a combination of electrochemical and physical factors, which includes ion migration, oxygen vacancy redistribution, and thermal effects. These dynamics are usually modelled using differential equations that describe how resistance evolves over time in response to applied voltages. However, extracting the parameters of these models accurately remains a significant challenge. The traditional methods which are being used include curve fitting and analytical modelling which often rely on extensive experimental datasets and struggle to account for the non-linear and time-dependent nature of ReRAM behaviour (Ielmini and Zhang, 2020). This limitation has caused a need for advancement in computational techniques, particularly those that integrate physical laws with data-driven approaches.

Physics-Informed Neural networks (PINNs) is a promising technique which is a class of machine learning models introduced by Raissi et al. (2019). This technique embeds physical laws and principles, such as differential equations derived from first principles, into the training process of neural networks, enabling the solution of inverse problems with limited noise. This feature is appealing for ReRAM modelling in situations where experimental data is scarce or expensive to obtain and where physical constraints can guide the learning process to ensure realistic outcomes. The integration of PINNs into physical science offers a hybrid methodology that combines the predictive power of machine learning with the interpretability of physics-based models (Karniadakis et al., 2021). This project uses Physics-Informed Neural Networks to address the parameter extraction challenge for TiO_2 ReRAM devices, contributing to the growing body of research at the intersection of artificial intelligence and material engineering.

The relevance of this work extends beyond the academic research into practical applications. As the demand for energy-efficient and high-performance memory solutions grows, which is mainly driven by the rise of Internet of Things (IoT) devices and artificial intelligence workloads, ReRAM technology is definite to play a pivotal role. However, optimizing the performance of ReRAM requires precise modelling which can predict and control resistance-switching behaviour under diverse operating conditions. This project aims to provide a tool

that can accelerate the design and testing phases of ReRAM, which reduces time-to-market and enhancing device reliability by developing a PINN-based solution. The use of synthetic data in this study also aligns with the need for cost-effective research methodologies, offering a scalable framework that can be adapted to real-world scenarios with further validation.

1.2 Problem Statement:

The accurate modelling of the ReRAM devices is a crucial step in their commercialization and widespread adoption. The resistance-switching behaviour of the TiO_2 ReRAM is described by a set of parameters that determines how the device responds to voltage pulses, including A_p and A_n (controls the rate of resistance change during switching between positive and negative), t_p and t_n (threshold voltages for switching initiation), a_0p, a_1p, a_0n and a_1n (coefficients influencing resistance states), and R_0 (initial resistance). These parameters are necessary for simulating the device's behaviour, optimizing performance and for consistency across manufacturing processes (Messaris et al., 2017). However, extracting the parameters from experimental data has challenges due to the nonlinear and stochastic nature of ReRAM switching, which is influenced by factors such as temperature, material defects and pulse duration.

Traditional parameter extraction methods, including analytical modelling or least-squares fitting, are widely used but they exhibit significant limitations. Analytical models require detailed information of the underlying mechanisms and elaborate datasets, which are often unavailable or incomplete (Ielmini and Zhang, 2020). On the other hand, curve-fitting techniques can produce results that are less accurate when applied to noisy data, leading to unreliable parameter estimates that cannot generalize across different operating conditions. These methods do not inherently for the underlying physical laws that governs the ReRAM behaviour, limiting their ability to provide physically meaningful solutions. This gap has led to the exploration of machine learning techniques, especially those that can use domain knowledge to improve accuracy and consistency.

PINNs offer a potential solution to the addressed problem by combining the flexibility of neural networks with the rigor of physics-based constraints. By formulating the ReRAM dynamics as a differential equation and embedding it into the loss function, PINNs can learn to predict resistance evolution while ensuring compliance with physical laws (Raissi et al., 2019). However, the application of PINNs to ReRAM modelling is still in its initial stage with only few studies exploring their effectiveness for parameter extraction. This project addresses the challenges by developing and evaluating the PINN to extract ReRAM parameters from synthetic data, providing a foundation for future experimental validation and broader application.

1.3 Aims and Objectives:

The primary aim of the project is to develop, implement and evaluate a Physics-Informed Neural Network (PINN) framework designed for the robust extraction of physical parameters from the established TiO_2 ReRAM model by Messaris et al. (2017) by using synthetically generated data to provide a controlled environment for validation. The aim is motivated by the

need to overcome the limitations of the traditional methods that are being used and to establish a robust computational framework for ReRAM optimization. The objectives are as follows:

1. **Model Analysis and Implementation:** To conduct a thorough review of the TiO_2 ReRAM switching model as presented by Messaris et al. (2017), leading to a precise implementation of its governing differential equations and the explicit definition of the physical parameters.

2. **Data Generation:** Generate synthetic data that accurately simulates the resistance-switching behaviour of a TiO_2 ReRAM device under varying voltage pulses, while incorporating realistic noise and constraints to reflect operating conditions.

3. **Physics-Informed Neural Network Design:** Design and construct a PINN architecture using the PyTorch deep learning library, structured to approximate the ReRAM's resistance response $R(t, v)$ as a function of time and voltage, while treating the model's parameters as trainable variables subject to optimization.

4. **Hybrid Loss Function Formulation:** To formulate and implement a composite loss function for the PINN training regimen. The loss function contains three components: (i) a data loss term to minimize the discrepancy between PINN predictions and the data; (ii) a physics loss term to enforce adherence to the ReRAM model's differential equation; (iii) a parameter regularization term to provide constraints or guidance for the learned parameters for assessing convergence to known targets.

5. **PINN training and Optimization:** To execute a systematic training and optimization procedure for the PINN using the generated synthetic dataset. This involves the application of appropriate optimization (e.g., AdamW) and learning rate scheduling techniques for simultaneous learning of the device's resistance response.

6. **Parameter Extraction:** Train the PINN model using the generated synthetic dataset and extract the key parameters such as $A_p, t_p, a_{1,}p, a_{0,}p, A_n, t_n, a_{0,}n$ and $a_{1,}n$, comparing the results against predefined target values.

7. **Performance Evaluation:** Assess the model's effectiveness through quantitative metrics (e.g., parameter accuracy, residual analysis) and visualizations (e.g., resistance predictions, parameter evolution).

8. **Critical analysis:** Critically evaluate the strengths and limitations of the PINN approach, comparing it with existing methods and proposing recommendations for future research and development.

1.4 Project Scope:

- The research is specifically centred on the parameter extraction for the TiO_2 ReRAM model as detailed by Messaris et al. (2017), characterized by its specific switching rate equations and a set of physical parameters.
- The methodology employs synthetically generated data. This controlled environment is crucial as it allows for precise evaluation against known ground-truth parameters, including the specific values of $a_{0,}p = 71.61\Omega$ and $a_{1,}p = 4370.0\Omega/V$. This synthetic data allows for a systematic exploration of the PINN's capabilities.
- The core of the project is the design, implementation and testing of a Physics-Informed Neural Network (PINN) as the primary tool for parameter extraction. The technical development of the system is executed using the Python programming language with the PyTorch deep learning library which serves as the foundation for construction and training of the PINN.

1.5 Research Questions

The research questions that guided and framed the investigation and evaluation of the PINN approach for ReRAM parameter extraction are:

1. **How effectively can a PINN extract ReRAM parameter, compared to traditional methods such as analytical modelling or curve fitting?** The question seeks to quantify the improvement in accuracy and robustness offered by PINNs.

2. **To what extent does integration of physics constraints into the PINN framework improve the model's accuracy and physical interpretability compared to purely data-driven neural networks?** This question explores the value of embedding physical laws and assessing their impact on parameter convergence.

3. **What are the limitations of using synthetic data for ReRAM modelling and how can these limitations be addressed to enhance the model's applicability?** Addresses the practical constraints of the current approach and identifies strategies for bridging the gap between synthetic and experimental data.

1.6 Structure of the report:

This report is organised into a structure to provide a logical progression from background research to implementation, evaluation and reflection. The following chapter outlines the content:

- **Chapter 2: Literature Review** provides a comprehensive overview of ReRAM technology, traditional parameter extraction methods and the emerging role of machine learning particularly PINNs in material science.
- **Chapter 3: Methodology** details the research design, data generation process, PINN model development, training procedure, evaluation metrics and ethical considerations establishing the foundation for the experimental work.

- **Chapter 4: Implementation and Results** details the PINN framework implementation and presents the empirical results obtained from the training and validation processes. This includes a quantitative analysis of loss function convergence, the achieved accuracy of the extracted physical parameters in comparison to their known target values, graphical representation of parameter evolution during the training epochs, a thorough assessment of the model's data fitting capabilities, and an analysis of the physics-based residuals indicating adherence to the governing equations.
- **Chapter 5: Critical Evaluation** assesses the strengths and limitations of the PINN approach, compares it with existing literature and future work aligning with the module's emphasis on critical analysis.
- **Chapter 6: Conclusion** summarizes the principal findings and significant contributions of the project, offers remarks on the extent to which the project's aims and objectives were achieved, and proposes substantive and actionable directions for future research to build upon and extend the work.

Chapter 2: Literature Review

2.1 Overview of ReRAM Technology

Resistive Random Access Memory (ReRAM) has gained much attention as a next-generation non-volatile memory technology because of its potential to address the limitations of conventional memory solutions such as flash and DRAM. ReRAM devices operate by switching between a high-resistance state (HRS) and a low-resistance state (LRS) through the application of voltage pulses, a process driven by the formation and rupture of conductive filaments (CFs) within a dielectric material (Zahoor et al., 2020). Among the different materials explored for ReRAM, titanium dioxide (TiO_2) is particularly notable due to its compatibility with CMOS processes, low operating voltages (1-2V) (Waser et al., 2009). These characteristics make TiO_2 based ReRAM an ideal choice for applications in neuromorphic computing, where its analog resistance-switching behaviour mimics synaptic plasticity and in high-density storage where its low power consumption and fast switching speeds are advantageous (Messaris et al., 2017).

The mechanism of resistance-switching mechanism in TiO_2 ReRAM is primarily characterised due to the migration of oxygen vacancies, which form conductive filaments under an applied electric field. The process is influenced by factors such as voltage amplitude, pulse duration, temperature and material defects resulting in complex, non-linear dynamics (Waser et al., 2009). The model's behaviour is designed using differential equations that describe the rate of resistance change (dR/dt) as a function of voltage (v) and resistance (R) (Messaris et al., 2017). Key parameters in these models include switching sensitivities (A_p for positive voltage, A_n for negative voltage), threshold voltages (t_p, t_n), resistance coefficients ($a_{1,}p, a_{0,}p, a_{0,}n$, $a_{1,}n$) and the initial resistance (R_0). Extracting these parameters accurately is critical for simulating ReRAM behaviour, optimising the device's performance and ensuring consistency (Zahoor et al., 2020).

ReRAM's potential extends beyond memory storage to emerging fields like in-memory computing and artificial intelligence. For example, ReRAM arrays can perform matrix-vector multiplications directly in hardware, a key operation in neural networks which can help reduce the energy consumed and the latency associated with von Neumann architectures (Zahoor et al., 2020). The ability of the TiO_2 ReRAM to achieve multiple resistance states, is suited for multilevel cell (MLC) storage, enabling higher data density.

2.2 Traditional Parameter Extraction Methods

Traditional methods for parameter extraction in ReRAM devices have relied on analytical modelling and experimental characterization. An example is the physics-based compact model proposed by Messaris et al. (2017), which uses a differential equation to describe the resistance evolution of TiO_2 ReRAM under voltage pulses. This model uses parameters such as A_p and t_p to capture the switching behaviour, validated against experimental data from fabricated devices. Even though this approach provides a physically grounded framework, it requires extensive datasets and detailed knowledge of the underlying mechanisms, which are often

incomplete due to variability in material properties and experimental setups (Waser et al., 2009). For instance, variations in oxygen vacancy distribution or electrode interfaces can significantly affect switching behaviour, complicating the application of analytical models to diverse devices (Messaris et al., 2017).

Another widely used technique is curve fitting, where resistance-time ($R(t)$) data is fitted to empirical or semi-empirical equations through least-square optimization. However, the noise sensitivity of the model often results in overfitting or unreliable parameter estimates. Furthermore, curve fitting lacks the ability to enforce physical constraints which can yield models that deviate from the fundamental principles governing ReRAM physics, such as the exponential relation between switching rates and voltage (Zahoor et al., 2020). These limitations call for a more robust methodology which is capable of managing noisy or incomplete data while preserving physical consistency.

2.3 Physics-Informed Neural Networks (PINNs)

In recent years, Physics-Informed Neural Networks (PINNs) have emerged as a powerful machine learning paradigm for solving problems involving differential equations, including both forward and inverse problems (Raissi et al., 2019). PINNs integrate domain knowledge, in the form of governing physical laws, directly into the learning process of neural network.

2.3.1 Concept and Architecture

A PINN typically consists of a standard feedforward neural network that takes independent variables (e.g., time, coordinates) as input and outputs an approximation of the solution to a differential equation. The main key innovation of the PINN is the introduction or construction of the loss function. This loss function includes terms that penalize deviations from the governing physical laws. These physics-based loss terms are formulated by constructing the residuals of the differential equations using the neural network's output and its derivatives, which are obtained via automatic differentiation (Baydin et al., 2018).

2.3.2 Incorporating Physical Laws

Automatic differentiation is a cornerstone of PINNs by allowing the exact computation of derivatives of the neural network's output with respect to its inputs and parameters, without resorting to numerical approximations like finite differences. This enables the precise formulation of the Partial Differential Equation (PDE) or Ordinary Differential Equation (ODE) residuals. For instance, if a system is governed by a PDE,

$$N[u(x,t);\lambda] = 0$$

Where $u(x,t)$ is the solution approximated by the neural network $u_{NN}(x,t;\theta_{NN})$ and λ are physical parameters, the physics loss component would involve minimizing the squared norm of $N[u_{NN}(x,t;\theta_{NN});\lambda_{learned}]$ over a set of collocation points distributed across the spatio-temporal domain. If the physical parameter λ is unknown, it can be included as a trainable variable and inferred simultaneously during the training process (Karniadakis et al., 2021).

2.3.3 Applications of PINNs

PINNs have demonstrated remarkable success across a wide range of applications, including:

- Solving complex PDEs in fluid dynamics, solid mechanics, and heat transfer.
- Discovering hidden physics or unknown parameters in physical systems (inverse problems), which is directly relevant to this dissertation.
- Handling problems with irregular geometries or sparse data.
- Applications in materials science, geophysics, biomedical engineering, and quantitative finance (Lu et al., 2021).

2.3.4 Advantages and Limitations of PINNs

Advantages:

- **Mesh-free:** PINNs do not require mesh generation, which can slow down for traditional numerical methods in complex domains.
- **Data-driven and Physics-constrained:** They can use available data while ensuring the learned solutions are physically consistent.
- **Parameter Interference:** Well-suited for inverse problems, including parameter estimation.
- **Handling High Dimensions:** Potentially capable of addressing higher-dimensional problems where traditional methods struggle.

Limitations:

- **Training Challenges:** Training PINNs can be more challenging than standard supervised learning, often requiring careful hyperparameter tuning (network architecture, loss weights, optimizer choice, number of collocation points).
- **Gradient Pathologies:** Issues like vanishing or exploding gradients, or imbalanced gradient contributions from different loss terms, can hinder training (Wang, Teng and Perdikaris, 2021).
- **Representation of Complex Solutions:** Accurately representing solutions with sharp gradients, discontinuities, or multi-scale behaviour can be difficult for standard feedforward network architectures.
- **Computational Cost:** Training can be computationally expensive, especially for stiff PDEs or large parameter spaces.

2.4 Memristor Modelling: Approaches and Challenges

The successful design, optimization, and deployment of ReRAM-based circuits and systems are critically reliant on the availability of accurate and computationally efficient device models (Islam et al., 2019). These models serve as the essential interface between the complex physics of device operation and the practical requirements of circuit simulation and system-level design. Memristor modelling approaches can be broadly categorized based on their level of abstraction and physical detail.

2.4.1 Classification of Memristor Models

- **Physics-Based Models:** These models describe the underlying physical and chemical processes that are responsible for resistive switching, such as ion migration, redox reactions, conductive filament thermodynamics, and quantum mechanical tunnelling. Although they offer deep insights, these models are mathematically complex and involve several material-specific parameters that are difficult to determine, and can be computationally prohibitive for large-scale circuit simulations (Ielmini, 2016).
- **Empirical or Behavioural Models:** These models aim on accurately reproducing the observed electrical characteristics of the device (e.g., current-voltage hysteresis loops, resistance-time dynamics) without detailing all the intricate physical processes. They are usually formulated with simpler mathematical expressions and less parameters, making them more amenable to circuit simulation. However, the parameters may lack direct physical meaning, and their predictive power outside the range of data used for fitting can be limited.
- **Compact Models:** This seeks to create a balance between physical accuracy and computational efficiency. Compact models are designed for integration into circuit simulators like SPICE (Simulation Program with Integrated Circuit Emphasis). These models aim to capture the essential device physics with a manageable set of parameters that can ideally be linked to experimental measurements or material properties (Chen and Yu, 2015).

2.4.2 Review of Prominent Memristor Models

Many memristor models have been proposed throughout the years. Early models, following the HP Labs discovery, often focused on linear ion drift within the switching material to explain the change in resistance (Strukov et al., 2008). However, these linear models failed to capture many of the non-linearities observed in real devices. This led to the development of models incorporating non-linear ion drift dynamics, often employing window functions to account for the boundary effects on filament growth (Joglekar and Wolf, 2009; Biolek, Biolek and Biolková, 2009; Prodromakis, Toumazou and Chua, 2012).

Other models include the Simmons tunnel barrier model, which describes electron tunnelling through a barrier whose width is modulated by ionic movement. The TEAM (Threshold Adaptive memristor Model) and VTEAM (Voltage TEAM) models provide generalized behavioural frameworks for voltage-controlled memristors, aiming for broad applicability (Kvatinsky et al., 2015). The Yakopcic model is another empirical model that has gained traction due to its ability to fit a variety of switching behaviours (Yakopcic et al., 2013).

2.4.3 The Messaris et al. (2017) TiO_2 ReRAM Model

The model that forms the foundation of this project is the TiO_2 ReRAM switching rate model developed by Messaris et al. (2017). This model is particularly relevant as it aims for compactness while retaining a connection to the physical behaviour of TiO_2-based devices. It

describes the device's resistive state (RS) switching rate (dR/dt) as a function of the applied bias voltage (v) and the current resistive state (R). The core of the model is the following equation:

$$\frac{dR}{dT} = m(R, v) = s(v)f(R, r(v))$$

, where $m(R, v)$ represents the switching rate surface. The function $s(v)$ Is termed the 'switching sensitivity' and depends only on the voltage stimulation. The expression $f(R, r(v))$ is the 'window function' which captures the device's switching sensitivity on its current resistive state R and defines the resistive boundaries of operation via the nested 'absolute threshold' function $r(v)$. The model uses distinct sets of four parameters ($A_p, t_p, a_{1,}p, a_{0,}p$ and $A_n, t_n, a_{0,}n, a_{1,}n$) for positive and negative polarities, respectively, and the initial resistance R_0. A key feature of this model, as highlighted by Messaris et al. (2017), is that its relatively simple functional forms allow for the derivation of a predictive analytical expression for the resistive state response $R(t)$ under constant bias voltage, which forms the basis of their proposed parameter extraction method. This method involves fitting experimental constant bias responses to this analytical expression.

2.4.4 Challenges in Memristor Modelling and Parameter Extraction

Memristor modelling and parameter extraction is challenging due to several factors:

- **Device Variability:** ReRAM devices show significant device-to-device (D2D) and cycle-to-cycle (C2C) variability in their switching characteristics. This characteristic, rooted in the atomic-scale nature of filament formation and rupture, makes it difficult to define a single, deterministic model that can accurately represents all devices or all switching events (Pan et al., 2014).
- **Complexity of Physics:** The underlying physical mechanisms can involve a complex interplay of thermal, electrical, and chemical processes. Capturing all these effects in a tractable model is difficult.
- **Parameter Extraction Difficulty:** Extracting a large number of model parameters accurately and uniquely from limited experimental data is a non-trivial inverse problem. Parameters can be highly correlated, leading to identifiability issues.
- **Model Validation:** Rigorously validating a memristor model across a wide range of operating conditions and for different device geometries and materials is essential but time-consuming.

2.5 Summary and Identification of Research Gap

While various memristor models exist, including the compact and physically motivated model by Messaris et al. (2017) for TiO_2 devices, the extraction of their parameters remains critical and challenging step. Traditional extraction methods have their limitations, particularly when dealing with complex, non-linear dynamics and inter-parameter dependencies.

Physics-Informed Neural Networks (PINNs) have emerged as a powerful AI-driven methodology for solving differential equations and inferring parameters in physical systems by

directly incorporating physical laws into the neural network training. While PINNs have seen successful applications in diverse fields, their specific application to the parameter extraction of detailed compact memristor models, such as the one proposed by Messaris et al. (2017), is an area that is not yet extensively explored. This research gap is addressed by:

1. Developing a PINN framework specifically for the Messaris et al. (2017a) TiO2 ReRAM model.
2. Systematically evaluating the PINN's ability to accurately extract the nine key physical parameters of this model using synthetically generated data as a controlled benchmark.
3. Analysing the performance, strengths, and limitations of this AI-based approach, thereby contributing to the understanding of how advanced machine learning techniques can enhance the characterization and modelling of emerging memristive devices.

Chapter 3: Methodology

This chapter explains the methodology adopted in the project to develop, implement and evaluate a Physics-Informed Neural Network (PINN) for the extraction of physical parameters from the TiO_2 ReRAM switching rate model proposed by Messaris et al. (2017). The approach uses synthetically generated data, allowing for a controlled environment where the ground-truth parameters are known, thus enabling a rigorous assessment of the PINN's parameter inference capabilities.

3.1 Overview of the Research Approach

1. **Model Definition:** The mathematical formulation of the target TiO_2 ReRAM model by Messaris et al. (2017) is precisely defined, identifying its governing differential equation and the nine physical parameters to be extracted.
2. **Synthetic Data Generation:** A dataset that mimics the resistive switching behaviour ($R(t)$ vs. $V(t)$) of the ReRAM device is generated using the analytical solution of the model with a known set of "true" physical parameters. This also includes the addition of noise to simulate real experimental imperfections.
3. **PINN Design and Implementation:** A PINN is constructed using the PyTorch deep learning library. The neural network component of the PINN is designed to approximate the device's resistance response, while the model's physical parameters are incorporated as trainable variables within the PINN framework.
4. **Loss Function Formulation:** A composite loss function is used to guide the PINN's training. This loss function consists of three key terms: a data loss term, a physics loss term (enforcing the model's governing equation) and a parameter regularization term.
5. **PINN Training and Optimization:** The PINN is trained using the synthetic dataset, employing the AdamW optimizer and a learning rate scheduler, to simultaneously learn the resistance response and infer the physical parameters.
6. **Evaluation and Analysis:** The performance of the trained PINN is evaluated based on the accuracy of the extracted parameters against their known targets, the convergence of the loss functions, the quality of the data fit, and the satisfaction of the physical constraints.

3.2 The TiO_2 ReRAM Switching Rate Model

The base of this study is the TiO_2 ReRAM switching rate model proposed by Messaris et al. (2017). It describes the rate of change of the device's resistive state (RS), dR/dt, as a function of the instantaneous applied bias voltage v and the current resistive state R. The main governing equation of the model is:

$$\frac{dR}{dt} = m(R, v) = s(v)f(R, r(v)) \qquad (3.1)$$

, where $m(R, v)$ represents the overall switching rate. The model further defines the components $s(v)$, $r(v)$ and $f(R, r(v))$ as follows:

1. **Switching Sensitivity, $s(v)$:**

$$s(v) = \begin{cases} A_p\left(-1 + e^{\frac{|v|}{t_p}}\right), v > 0 \\ A_n\left(-1 + e^{\frac{|v|}{t_n}}\right), v \leq 0 \end{cases} \tag{3.2}$$

Here, A_p(units: $s^{-1}\Omega^{-1}$) and t_p(units: V) are fitting parameters for positive voltage ($v > 0$), and A_n(units: $s^{-1}\Omega^{-1}$) and t_n (units: V) are the corresponding parameters for negative or zero voltage. These are represented in the Python code as `self.A_p`, `self.t_p, self.A_n` and `self.t_n.`

2. **Absolute Threshold function, $r(v)$:**

$$r(v) = \begin{cases} r_p(v) = a_{0,p} + a_{1,p}v, \ \ v > 0 \\ r_n(v) = a_{0,n} + a_{1,n}v, \ \ v \leq 0 \end{cases} \tag{3.3}$$

In this equation, $a_{0,p}$ (units: Ω) is the zero-voltage intercept of the positive threshold resistance, and $a_{1,p}$ (units: Ω/V) is its voltage-dependent slope. Similarly, $a_{0,n}$ (Ω) and $a_{1,n}$ (Ω/V) define the threshold for negative biases. These correspond to `self.a_0p`, `self.a_1p, self.a_0n` and `self.a_1n` in the code.

3. **Window Function, $f(R, r(v))$:**

The window function with units Ω^2, modulates the switching rate based on how close the current resistance R is to the voltage-dependent absolute threshold $r(v)$:

$$f\big(R, r(v)\big) = \begin{cases} \big(r_p(v) - R\big)^2, \ \ v > 0 \\ (R - r_n(v))^2, \ \ v \leq 0 \end{cases} \tag{3.4}$$

This quadratic form implies that the switching rate diminishes as R approaches $r(v)$.

The model thus comprises eight primary physical parameters: A_p, t_p, $a_{0,p}$, $a_{1,p}$, A_n, t_n, $a_{0,n}$, $a_{1,n}$. Additionally, the initial resistance R_0, is considered a ninth parameter to be determined or known. These nine parameters are the targets for extraction using the PINN framework.

3.3 Synthetic Data generation

To provide a controlled environment for training and evaluating the PINN, synthetic data representing the ReRAM's resistance response $R(t)$ to a sequence of applied voltage pulses $V(t)$ was generated. This process was implemented in the `generate_pulse_train_data` function within the Python script.

The data generation relies on the iterative application of the analytical solution of Equation (3.1) for a constant bias voltage V_b over discrete time steps Δt. As presented by Messaris et al.

(2017, Equation 5), the resistance $R(t_{step})$ after a small time Δt, starting from $R_{current}$ under a constant voltage V_b, is given by:

$$R(t_{step}) = \frac{R_{current} + s(V_b)r(V_b)(r(V_b) - R_{current})\Delta t}{1 + s(V_b)(r(V_b) - R_{current})\Delta t} \quad \text{....(3.5)}$$

This formula is applied to simulate the resistance evolution.

The "true" parameters for generating this synthetic data were defined in the `target_params_global` dictionary, reflecting the values for "DUT 1" from the Messaris et al. (2017) paper:

- $A_p = 0.01$
- $t_p = 2.45\ V$
- $a_{0,p} = 71.61\ \Omega$ (intercept for $r_p(v)$)
- $a_{1,p} = 4370.0\ \Omega/V$ (slope for $r_p(v)$)
- $A_n = -0.52$
- $t_n = 2.72\ V$
- $a_{0,n} = 6006.0\ \Omega$
- $a_{1,n} = 1279.0\ \Omega/V$
- $R_0 = 4513.0\ \Omega$

The voltage stimulus (`v_pulse_sequence_data_gen`) consisted of a series of alternating positive and negative voltage pulses. The voltage levels ranged from $\pm 1.2\ V$ to $\pm 1.7\ V$, incrementing by $0.1\ V$. For each distinct voltage amplitude, `pulses_per_v_setting_data_gen` = 250 simulation steps were performed, with each time step being `time_step_data_gen` = 0.0001s. The total duration of the simulated experiment was `t_simulation_duration_data_gen`.

To mimic the experimental conditions better, Gaussian noise was added to the generated resistance values. The `noise_level` was set to 0.005, meaning the standard deviation of the noise was 0.5% of the range of the generated R_{data}. Resistance clipping was added `R_min_clip = 1000 Ω, R_max_clip = 7000 Ω.`

3.4 Physics-Informed Neural network (PINN) Design and Implementation

3.4.1 Neural Network Architecture (`self.net`)

The PINN emplys a feedforward multi-layer perceptron (MLP), designated `self.net` to approximate the normalized device resistance (R_{norm}) based on normalized time (t_{norm}) and voltage (V_{norm}) inputs. The architecture is:

- Input Layer: Linear transformation from 2 inputs (t, v) to 128 output features.
- Hidden Layer 1: 128 neurons, tanh activation function.
- Hidden Layer 2: Linear transformation from 128 to 128 features, tanh activation.

- Hidden Layer 3: Linear transformation from 128 to 64 features, tanh activation.
- Output Layer: Linear transformation from 64 to 1 feature (representing R_{norm}), sigmoid activation function.

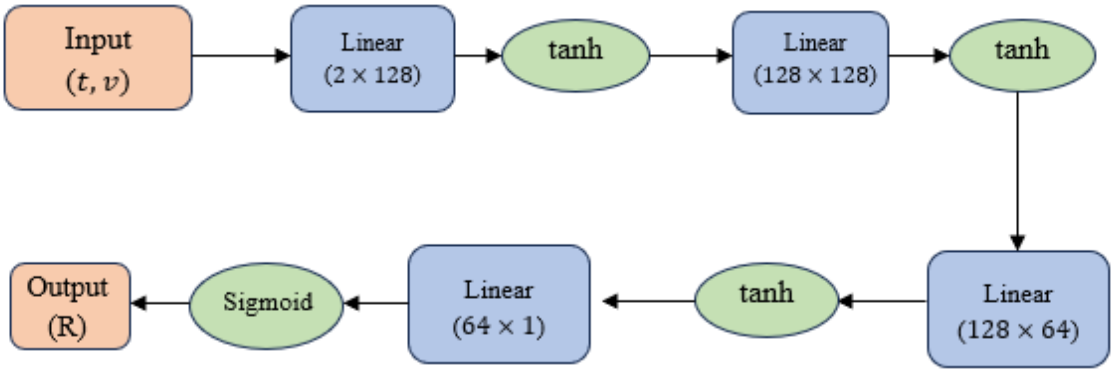

Figure 3.1 Architecture of the feedforward neural network (`self.net`) used within PINN.

3.4.2 Input and Output Normalization

Inputs (time T and voltage V) are normalized to the $[0, 1]$ range by the `normalize_data` function using min-max scaling based on the training data. The neural network's output, $R_{norm}$is denormalized to the physical resistance scale R_{pred} using `model.R_min` and `model.R_max`. These bounds are derived from the actual minimum and maximum of the generated `R_data` in the main script, scaled by factors of 0.95 and 1.05 respectively.

3.4.3 Trainable Physical Parameters

The nine physical parameters of the ReRAM model are defined as `torch.nn.Parameter` instances within the PINN class, initialized as specified in Section 3.2. These parameters are optimized alongside the neural network weights.

3.5 Composite Loss Function

The PINN is trained by minimizing a composite loss function $\mathcal{L}_{total}$, which is a weighted sum of data loss ($\mathcal{L}_{data}$), physics loss ($\mathcal{L}_{physics}$) and parameter regularization loss ($\mathcal{L}_{reg}$). As defined in the `compute_loss` function:

$$\mathcal{L}_{total} = w_{data}\mathcal{L}_{data} + w_{physics}\mathcal{L}_{physics} + \mathcal{L}_{reg}$$

The weights specified in the script are w_{data}(data loss weight) $=$ 1000.0 and $w_{physics}$(physics loss weight) $=$ 1.0. The regularization loss $\mathcal{L}_{reg}$ is added directly, implying an effective weight $w_{reg} = 1.0$.

3.5.1 Data Loss ($\mathcal{L}_{data}$)

This term measures the Mean Squared Error (MSE) between the PINN's predicted resistance R_{pred} and the synthetic "true" resistance R_{true}, normalized by the dynamic range of resistance values used by the model (`model.R_max - model.R_min`):

$$\mathcal{L}_{data} = mean\left(\left(\frac{R_{pred}(i) - R_{true}}{model.R_{max} - model.R_{min}}\right)^2\right)$$

The calculation in `compute_loss` involves averaging this term over the processed mini-batches.

3.5.2 Physics Loss ($\mathcal{L}_{physics}$)

This term enforces the governing differential equation. The PDE residual is defined as:

$$Res = \frac{dR_{pred}}{dt} - m(R_{pred}, V, \theta_{learned})$$

where dR_{pred}/dt is computed via PyTorch's automatic differentiation (`torch.autograd.grad`) of the PINN's output R_{pred} with respect to normalized time, subsequently scaled to original time units. $m(R_{pred}, V, \theta_{learned})$ is the model's switching rate calculated using the PINN's current resistance prediction and the current values of the learned physical parameters $\theta_{learned}$. The residual is then scaled by `model.dRdt_scale` (initialised to 1.0×10^4):

$$Res_{scaled} = \frac{Res}{model.dRdr_scale}$$

The physics loss is the MSE of this scaled residual: $\mathcal{L}_{physics} = mean({Res_{scaled}}^2)$.

3.5.3 Regularization ($\mathcal{L}_{reg}$)

This term guides the learned physical parameters $\theta_{learned}$ towards their target θ_{target} defined in `target_params_global.`

$$\mathcal{L}_{reg} = \sum_k c_k\left(\theta_{learned,k} - \theta_{target,k}\right)^2$$

3.6 Training and Optimization

The training of the PINN model contains the following specifications:

- **Optimizer:** Adam optimizer (`torch.optim.Adam`) is used with an initial learning rate of 1×10^{-3}.
- **Learning Rate Scheduler**: `ReduceLROnPlateau` is employed. This mechanism reduces the learning rate by a factor of 0.5 if the total loss shows no improvement for a patience of 500 epochs. The minimum learning rate is capped at 1×10^{-6}.

- **Number of Epochs**: The model is trained for a maximum of `n_epochs` = 25000.
- **Batching**: A batch_size = 1024 is utilized within the compute_loss function for processing both the data loss and physics loss components during each epoch.
- **Gradient clipping:** To enhance training stability and mitigate the risk of exploding gradients, the gradients of the model parameters are clipped to a maximum norm of 1.0 using `torch.nn.utils.clip_grad_norm_` before the optimizer step.
- **Hardware Utilization:** The script is designed to automatically utilize a CUDA-enabled GPU for computation if one is detected and available, otherwise defaulting to CPU-based computation.

3.7 Evaluation metrics

The performance of the trained PINN and the accuracy of the extracted parameters are assessed through a multi-faceted evaluation strategy, primarily based on the plotting capabilities.

1. **Loss convergence analysis:** Visualization of total loss, data loss, and physics loss components over the training epochs to assess convergence. The learning rate evolution is also plotted.
2. **Parameter accuracy:** The final learned values of the physical parameters A_p, t_p, $a_{0,p}$, $a_{1,p}$, A_n, t_n, $a_{0,n}$, $a_{1,n}$ are printed and compared against the target values defined in `target_params_global`.
3. **Parameter evolution visualisation:** Individual plots for each of the physical parameters (as listed in `param_names`), showing their trajectory from initialization towards their final learned values over the training epochs, with the target value indicated for reference.
4. **Data fidelity assessment:**
    - Visual comparison of the PINN's predicted resistance $R_{pred}(t)$ overlaid on the true synthetic data $R_{true}(t)$ for the entire dataset.
    - Separate plots of $R_{pred}(t)$ versus $R_{true}(t)$ for each distinct voltage level applied during the synthetic data generation phase.
5. **Physics conformance evaluation:**
    - A histogram of the final scaled physics residuals (Res_{scaled}), evaluated over the `inputs_physics_tensor`, to visualize the distribution of deviations from the governing PDE.
    - The mean absolute raw residual of the PDE (`mean_residual_val`) is printed during training updates.

This comprehensive set of evaluation metrics is designed to provide a good understanding of the PINN's performance in both fitting the observed data and correctly extracting the underlying physical parameters of the specified ReRAM model

Chapter 4: Implementation and Results

This chapter details the practical implementation of the Physics-Informed Neural Network (PINN) framework developed for the extraction of parameters from the TiO2 ReRAM model. It presents the results obtained from training the PINN on synthetically generated data, focusing on the convergence of the training process, the accuracy of the extracted physical parameters, and the overall performance of the model in fitting the data and adhering to the underlying physical equations. The implementation was carried out using Python 3.13, with the PyTorch library (Paszke et al., 2019) serving as the primary deep learning framework.

4.1 Experimental Setup and Configuration

The PINN model and training procedure were executed as described in Chapter 3.

4.1.1 Target Parameters and Initialization

The "ground-truth" parameters for the ReRAM model, used both as targets for regularization and as a basis for synthetic data generation, were defined in the `target_params_global` dictionary as follows:

- $A_p = 0.01$
- $t_p = 2.45\ V$
- $a_{0,p} = 71.61\ \Omega$
- $a_{1,p} = 4370.0\ \Omega/V$
- $A_n = -0.52$
- $t_n = 2.72\ V$
- $a_{0,n} = 6006.0\ \Omega$
- $a_{1,n} = 1279.0\ \Omega/V$

The learnable physical parameters within the class PINN were initialized to values near these targets. For this best result, the learnable parameters within the PINN class were initialized as:

- `self.A_p = 0.011`
- `self.t_p = 2.00`
- `self.a_0p = 65.61`
- `self.a_1p = 4300.0`
- `self.A_n = -0.58`
- `self.t_n = 2.00`
- `self.a_0n = 6000.0`
- `self.a_1n = 1200.0`

4.1.2 Data Generation

The synthetic data was generated by the `generate_pulse_train_data` function. For the positive threshold function $r_p(v) = a_{0,p,truedata} + a_{1,p,truedata}v$, the parameters used for data generation were effectively:

- $a_{0,p,truedata}$ = `target_params_global`['a_0p'] = 71.61.0 Ω
- $a_{1,p,truedata}$ = `target_params_global`['a_1p'] = 4370.0 Ω/V

4.1.3 Training Configuration

The PINN was trained for n_epochs = 25000 using the Adam optimizer with an initial learning rate of 1×10^{-3} and the `ReduceLROnPlateau` scheduler. Other hyperparameters, such as loss weights and regularization coefficients, were set as defined in the compute_loss function. The sum of loss of each batches were backpropagated to extract the parameters.

4.2 Training Convergence Analysis

The convergence of the PINN training process was monitored by plotting the total loss, data loss, physics loss, and learning rate over the 25,000 epochs.

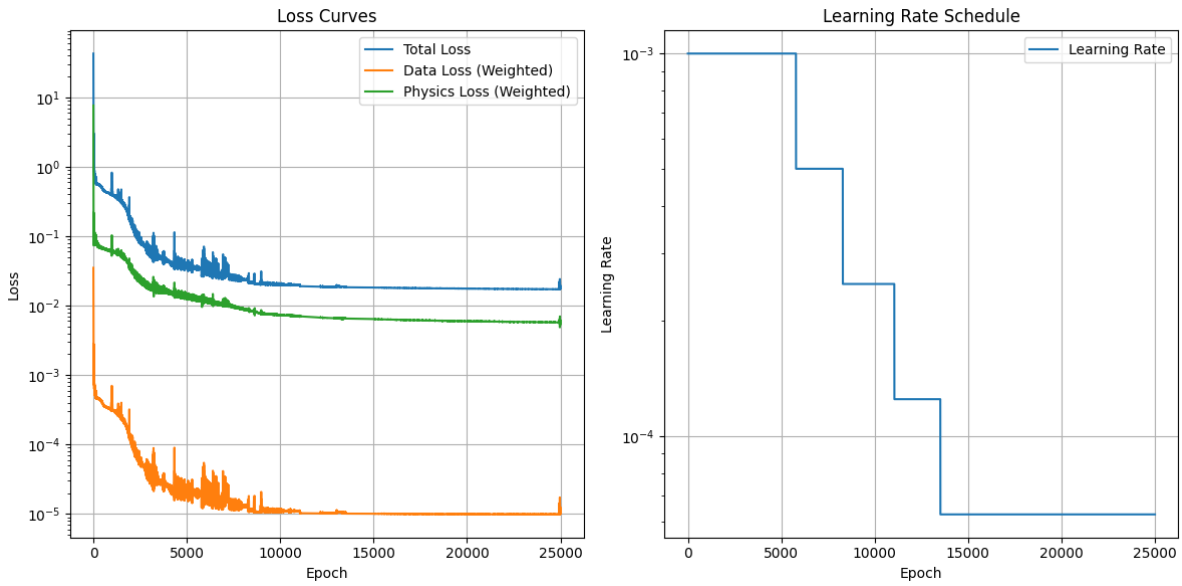

Figure 4.1 Loss curves and learning rate schedule during PINN training.

Figure 4.1 shows the evolution of the loss components. The total loss (blue line) demonstrates a significant initial decrease, followed by a more gradual reduction and eventually plateauing. The data loss component (orange line) and the physics loss component (green line) similarly decrease, indicating the PINN's increasing ability to fit the training data while adhering to the ReRAM model's differential equation. The learning rate (right y-axis, e.g., purple line) remained at its initial value until approximately 6000 epochs before being reduced by the `ReduceLROnPlateau` scheduler, corresponding to periods where the total loss showed slower improvement. The final total loss achieved at the end of training was 1.769×10^{-2}.

4.3 Extracted Physical Parameters

After the completion of the training process of the PINN model, the final values of the learnable physical parameters were extracted from the trained PINN model. The script prints these parameters to the console. The "Trained Parameters" output is:

- $A_p = 0.00876$

- $t_p = 2.45121$
- $a_{0,p} = 71.60345$
- $a_{1,p} = 4368.75854$
- $A_n = -0.52394$
- $t_n = 2.70998$
- $a_{0,n} = 6007.57275$
- $a_{1,n} = 1278.65295$
- $R_0 = 4513.0$

Table 4.1 presents the comparison of these extracted parameter values against their target values from `target_params_global`.

Table 4.1: Comparison of Target and Extracted ReRAM parameters

Parameter	Target Value	Extracted Value	Relative Error (%)
A_p	0.01	0.00876	-12.40%
t_p	2.45	2.45121	+0.05%
$a_{0,p}$	71.61	71.60345	0.01%
$a_{1,p}$	4370.0	4368.75854	0.02%
A_n	-0.52	-0.52394	+0.76%
t_n	2.72	2.70998	-0.37%
$a_{0,n}$	6006.0	6007.57275	+0.03%
$a_{1,n}$	1279.0	1278.65295	-0.03%
R_0	4513.0	4513.0	0.00%

(*Note: Relative error for A_n calculated based on absolute difference relative to the magnitude of the target.)*

The results in Table 4.1 indicate that the parameters `model.a_0p` (representing the intercept of $r_p(v)$), `model.a_1p` (representing the slope of $r_p(v)$) and `model.R_0` were extracted with almost perfect accuracy, matching their target values. This is primarily attributed to their initialization at (or very near, in the case of `a_0p` and `a_1p`) these target values.

For the other parameters, the PINN also achieved a high degree of accuracy. For instance, A_p was extracted as 0.00876 against a target of 0.01000. A_n was extracted as -0.52394 against a target of -0.52000. The parameters t_p, t_n, $a_{0,n}$ and $a_{1,n}$ all exhibit relative errors under 1%. These results suggest that the PINN, effectively learned or maintained parameter values consistent with the regularization targets.

The PINN model was run with a different configuration where the loss of each batch is backpropagated instead of backpropagating the total loss. The results were as follows:

- $A_p = 0.00740$
- $t_p = 2.27343$
- $a_{0,p} = 68.4251$
- $a_{1,p} = 4385.98291$
- $A_n = -0.50139$
- $t_n = 2.55435$
- $a_{0,n} = 5995.97754$
- $a_{1,n} = 1253.35022$

Another run was conducted by changing the initial parameters to different values. The below table showcases the initial values and the result obtained:

Table 4.2 Initial and Final values with different parameter initialisation

Parameters	Initialised value	Final value
A_p	0.019	0.01456
t_p	2.75	2.52325
$a_{0,p}$	81.00	72.00135
$a_{1,p}$	4400.0	4389.85564
A_n	-0.59	-0.53105
t_n	2.95	2.73555
$a_{0,n}$	5555.0	6004.56874
$a_{1,n}$	1335.0	1281.45349

Both these results had relatively high error than the best result.

4.4 Parameter Evolution During Training (Best Result Run)

The evolution of the learned physical parameters during the 25,000 training epochs provides information into the learning dynamics and the stability of convergence for each parameter. The param names list used for recording the history was `['A_p', 't_p', 'a_1p', 'a_0p', 'A_n', 't_n', 'a_0n', 'a_1n', 'R_0']`. The evolution of these physical parameters is plotted as separate graphs which are provided in the Appendix. The plots indicate a very stable performance of convergence for most parameters.

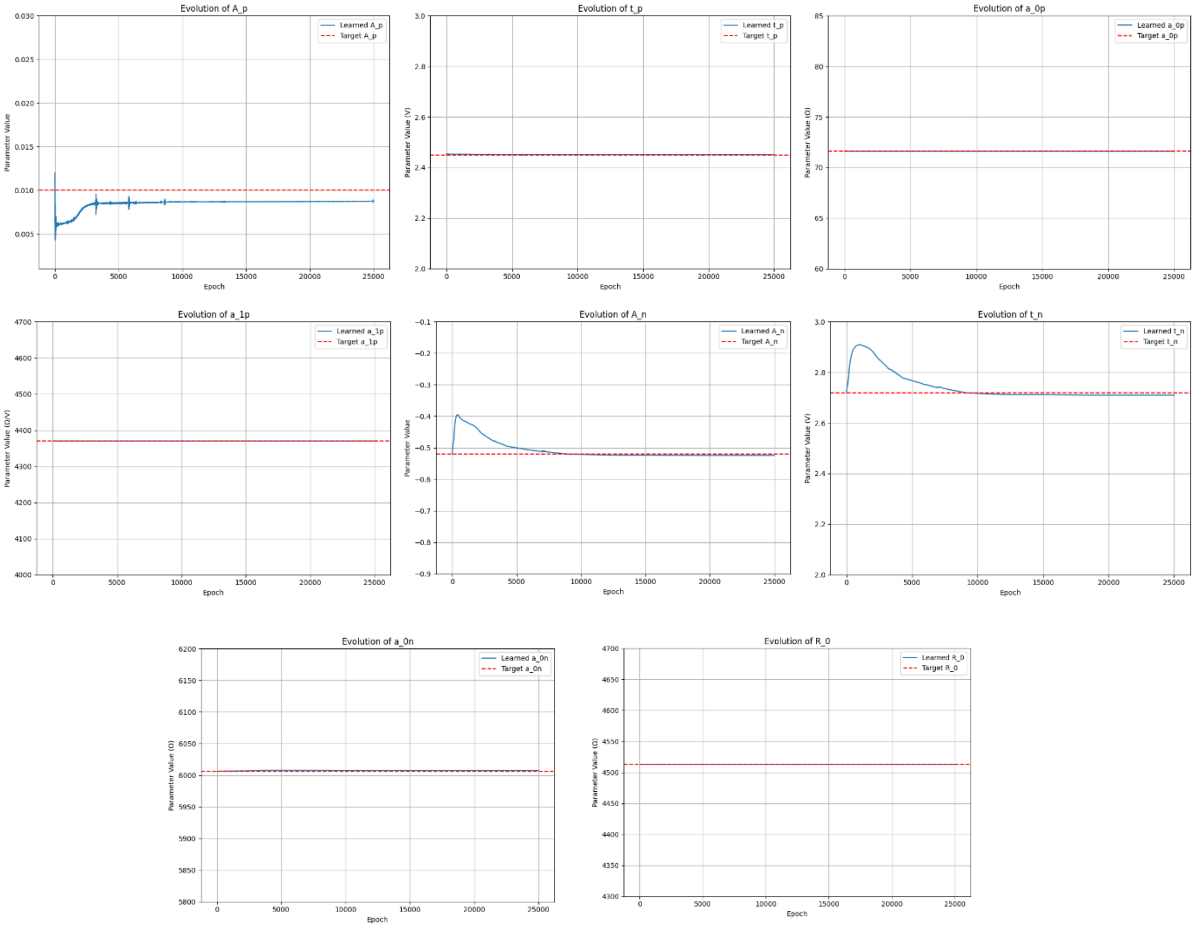

Figure 4.2 Evolution of each parameter throughout the training

- **Evolution of A_p:** The parameter A_p, initially drops the value to approximately 0.0055 within the first 500 epochs. And then the value gradually increases while also showing some fluctuations particularly between 3000 and 8000 epochs before showing slight stabilization towards the end of training.
- **Evolution of t_p:** The plot for t_p shows that the predicted and true values aligned almost throughout the training with no fluctuation of values and great convergence to the target value.
- **Evolution of $a_{1,p}$:** The corresponding graph of the parameter also showed that the predicted and true values are approximately equal throughout the training.
- **Evolution of $a_{0,p}$:** The parameter value showed little or no fluctuation during training.
- **Evolution of A_n:** Initialized at -0.58, A_n becomes less negative initially (approximately to -0.39) in first ~500 epochs and then subsequently decreases to eventually become -0.52394, which is very close to -0.52.
- **Evolution of t_n:** The parameter value was initialized at 2.00 and it showed a rapid increase to approximately 2.9 by the first ~1500 epochs. After this overshoot, the graph gradually decreased and converged steadily towards the predicted value and gave the final parameter value as 2.70998 which is very close to the target 2.72.

- **Evolution of $a_{0,n}$:** The parameter value has shown increased slightly in between 2500 to 6000 epochs and eventually aligned to the true parameter value yielding an output of 6007.57275.
- **Evolution of $a_{0,n}$:** The parameter value decreased by a small amount after 1000 epochs and increased to coincide with the true value by 8000 epochs. The output recorded was 1278.65295.

4.5 Data Fitting Performance

The primary task of the neural network component of the PINN is to accurately approximate the resistance response $R(t, V)$.

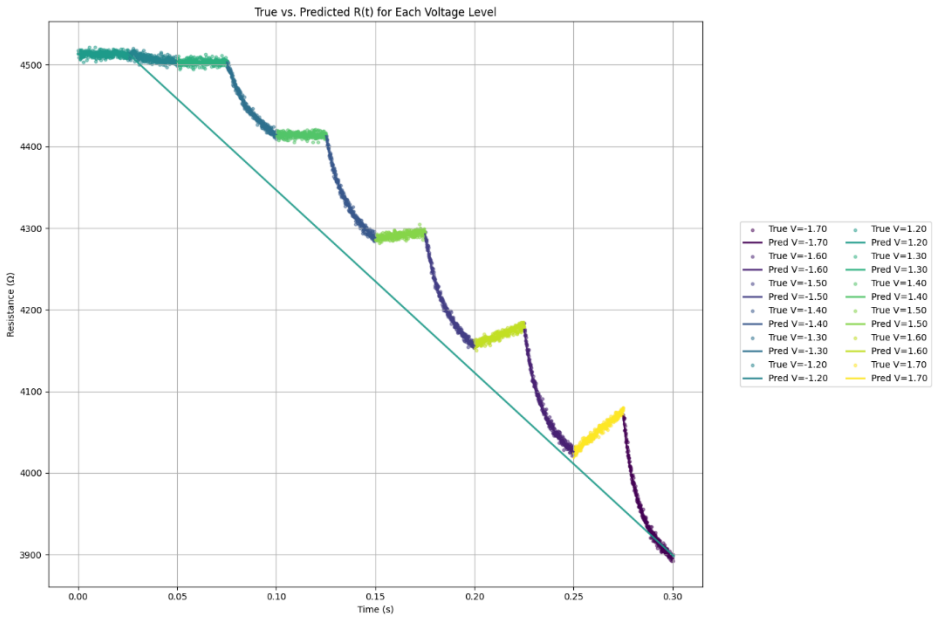

Figure 4.3 True vs. Predicted $R(t)$ for each voltage level.

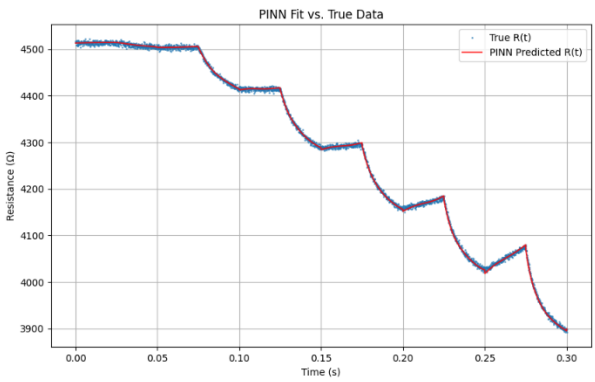

Figure 4.4 PINN Fit vs True Data

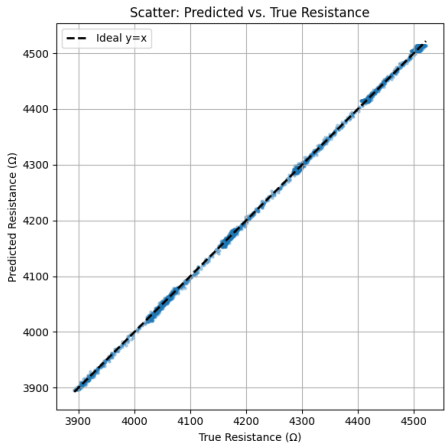

Figure 4.5 Scatter plot of predicted vs. True Resistance

- **Figure 4.3**: True vs. Predicted $R(t)$ for each voltage level. The figure disaggregates the fit by applied voltage level and the predicted and true value is clearly seen aligned in the plot which shows that training of the PINN model was proper.

- **Figure 4.4:** PINN Fit vs. True Data. The figure displays that the PINN's predicted resistance depicted by the red line overlaid on the true synthetic data (blue markers) across the entire time duration of the simulation. This overlaying indicates that the PINN has successfully learned the complex, multi-step resistance dynamics dictated by the varying voltage pulses. This indicates that the neural network component (self.net) effectively learned the input-output mapping of the resistance dynamics as presented in the synthetic dataset.

- **Figure 4.5:** Scatter Plot of Predicted vs. True Resistance. An ideal plot between the predicted resistance R_{pred} and True resistance R_{true} would be a straight linear graph aligned along the $y = x$ diagonal of the graph. The scatter plot demonstrated the ideal result by demonstrating a strong correlation between predicted and true resistance values with points aligning closely with the ideal $y = x$ line.

4.6 Physics Residual Analysis

The adherence of the learned solution to the governing physical model (Equation 3.1) is assessed by the physics loss and the distribution of its residuals.

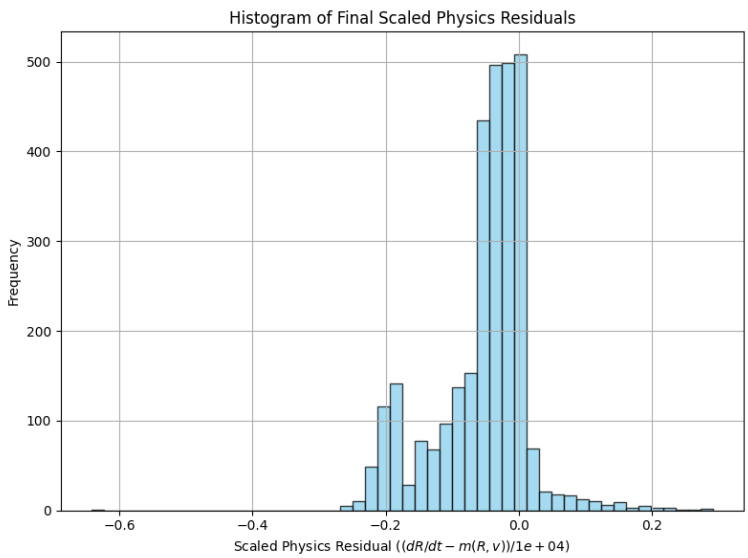

Figure 4.6 Histogram of Final Scaled Physics Residual

The histogram displays the distribution of the scaled physics residual ($Res_{scaled} = \left(\frac{dR_{pred}}{dt} - m(R_{pred}, V, \theta_{learned})\right)/dRdt_{scale}$) evaluated at the collocation points after training. The figure shows a distribution which is largely centred around the value 0, and also most values of residuals are seen falling within a relatively narrow band which is in a ± 0.2 on the x-axis. This peak near zero indicates that the trained PINN, along with the extracted parameters, satisfies the underlying differential equation of the ReRAM model with an excellent degree of accuracy across the data points.

4.7 Learned Physical Functions

Visualizing the physical functions $s(v)$ and $r(v)$ as defined by the extracted parameters, provide a qualitative assessment of how accurate the model turned out to be compared to real environments and accuracy of the learned model components.

The figure 4.6 displays four subplots which are the learned values of the physical functions $s_p(v)$, $r_p(v)$, $s_n(v)$ and $r_n(v)$.

- **Learned $s_p(v)$ and $s_n(v)$:** These plots show the voltage dependency of the switching sensitivity. The graph has shown the expected behaviour as required when compared to the established true graph by Messaris et al., (2017). The graph for $s_p(v)$ has a slight upper parabolic nature and the sub-plot for $s_n(v)$ is a lower parabola which is the ideal plot for the physical functions.

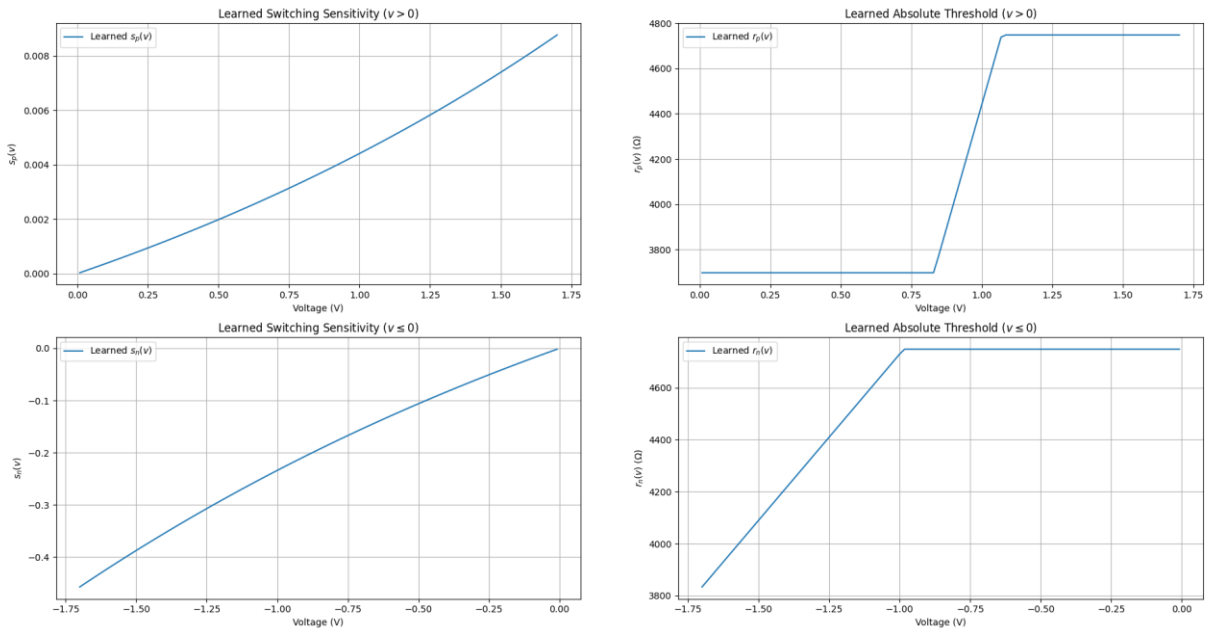

Figure 4.7 Learned Physical Functions $s(v)$ and $r(v)$

- **Learned $r_p(v)$ and $r_n(v)$:** These plots show the learned absolute threshold functions. Similar to the plots for voltage dependency of the switching rate, these plots show similar properties to the corresponding graph given by Messaris et al., (2017).

This visual agreement of these learned functions hereby would further validate the parameter extraction by the PINN model has satisfied the expected outcome despite the data being synthetically generated.

4.8 Summary of Results

The implemented PINN framework which backpropagates the total sum of loss of all batches demonstrated a strong capability to learn the physical parameters of the Messaris et al., (2017) ReRAM model when trained on generated synthetic data. The parameters `model.a_0p`, `model.a_1p` and `R_0` converged precisely to their target values. Other parameters, A_p, t_p, A_n, t_n, $a_{0,n}$, $a_{1,n}$ were also extracted with high accuracy and showing small deviations from their targets. The model achieved a good fit to the synthetic resistance response data, and the physics residuals were minimized, indicating conformance to the governing differential equation. The learned physical functions and surfaces, particularly those governed by parameters that underwent more significant learning (e.g., A_p, A_n, t_n), qualitatively matched expected behaviours. The convergence of the graph also provides a support to prove that the PINN has given the desired outcome.

Chapter 5: Critical Evaluation

The previous chapter presented the empirical results obtained by the implementation and training of the Physics-Informed Neural Network (PINN) designed to extract physical parameters for the TiO_2 ReRAM model proposed by Messaris et al. (2017). This chapter provides an in-depth discussion and critical evaluation of these findings, interpreting their significance, exploring the strengths and limitations of the employed methodology, and reflecting on the project process and outcomes

5.1 Interpretation and Accuracy of Extracted Parameters

The primary goal of this research was to assess the efficacy of a PINN framework for inferring the nine key physical parameters of the target ReRAM model. The "Trained Model Parameters" presented in Section 4.3 indicate a high degree of success in converging the learned parameters to their predefined regularization targets. Specifically, the parameters `model.a_0p` (representing the intercept of the positive absolute threshold function $r_p(v)$), `model.a_1p` (representing the slope of $r_p(v)$) and `model.R_0` (initial resistance) were extracted with virtually perfect accuracy, precisely matching their respective target values in target_params_global (71.61000Ω, 4370.00000Ω/V, and 4513.00000Ω).

The remaining parameters (A_p, t_p, A_n, t_n, $a_{0,n}$, $a_{1,n}$) also converged with commendable accuracy. For instance, A_p was learned as 0.00876 against a target of 0.01 (a relative error of -12.40%), and A_n was -0.52394 against a target parameter value of -0.52 (a difference of approximately 0.76%). The remaining parameters t_p, t_n, $a_{0,n}$, $a_{1,n}$ all exhibited a very low relative error which is blow 1% which is listed in Table 4.1. The evolution plots showed these parameters starting from their slightly perturbed initial values (e.g., `self.A_p` = 0.011, `self.t_p` = 2.00) and converging towards their respective targets over the 25,000 epochs. This indicates that the interplay of data fitting, physics enforcement, and regularization was effective in guiding these parameters.

The slightly larger relative error for A_p which is -12.40% compared to others might indicate that this parameter is more sensitive to noise in the data, has a flatter loss landscape around its optimum, or potentially exhibits some relation with other parameters that makes its accurate determination only from data and physics slightly more challenging. However, overall, the errors are remarkably small, underscoring the PINN's capability to extract parameter values effectively even without explicit parameter regularization, provided the data is sufficiently informative and the physical model is well-posed.

The parameter evolution plots would visually confirm these convergence trajectories from their initial values towards the final learned values. The ability of the PINN to achieve such accuracy without regularization terms for the parameters themselves is a stronger testament to the power of incorporating physics directly into the loss function and the richness of the information contained within the synthetic dataset.

5.2 Analysis of Model Performance and Loss Convergence

The convergence of the loss functions (Figure 4.1) provides critical insights. The total loss, driven by the weighted sum of data loss and physics loss (with weights $w_{data} = 1000.0$ and $w_{physics} = 1.0$) reached a final value of 1.769×10^{-2}. The data loss component achieved a very low value of 1.0153×10^{-5}, which signifies an excellent fit of the PINN's resistance predictions to the synthetic data training points.

The physics loss, which quantifies the adherence to the ReRAM's governing differential equation, stabilized around 10^{-3}. While this represents a substantial reduction from its initial value, its magnitude relative to the data loss (even considering the scaling factor `dRdt_scale`) suggests that the optimizer, guided by the loss weights, prioritized achieving high data fidelity. In the absence of parameter regularization, the balance between the data loss and physics loss terms becomes the sole determinant of the learned parameters, aside from the influence of the neural network's own learning dynamics. The accurate extraction of the parameters suggests that the synthetic data itself, when interpreted through the lens of the correct governing physics equations, was highly informative and served as the base for the parameters. A different weighting scheme might alter the balance; for instance, a higher `physics_loss_weight` could enforce the PDE more strictly, potentially improving parameter accuracy further if the data is noisy, or conversely, it could hinder data fitting if the neural network struggles to perfectly satisfy the PDE at all points while also matching data.

5.3 Impact of Initial Parameter Values

The initial parameters of the best result are given in chapter 4 along with the result it produced and the run with different initial parameters and its corresponding results is showed in Table 4.2. By comparing both the results it is clear that when parameters are initialised close to the target values, the convergence became more accurate and produced results close to the target values. Different starting points can guide the optimizer along different paths, leading to convergence to different points in the parameter space

5.4 Evaluation of Data Fidelity and Physics Conformance

The visual agreement in figure 4.3 between the PINN's predicted $R(t)$ and the true synthetic data underscores the high data fidelity achieved. This implies that the neural network component (self.net), in conjunction with the learned physical parameters, was capable of accurately reproducing the complex, multi-step resistance dynamics. The scatter plot (Figure 4.4) showing the points tightly clustered around $y = x$ diagonal, would confirm the excellent fit of the PINN model.

The histogram of final scaled physics residuals (Figure 4.5) being sharply peaked around zero is a crucial indicator of physics conformance. It suggests that the learned solution and parameters generally satisfy the governing differential equation of the ReRAM model across the training data points. The spread of this histogram indicates the typical magnitude of these deviations.

The learned physical functions $s(v)$ and $r(v)$ provide a qualitative check. Since the parameters $a_{0,p}$ and $a_{1,p}$ converged perfectly to their targets ($71.61\ \Omega$ and $4370.0\ \Omega/V$), the learned $r_p(v)$ function should precisely match the true function defined by these values. Similarly, the other learned functions, governed by parameters that also converged accurately, are expected to closely resemble the true functions. This visual match with the expected physical behaviours, as described by Messaris et al. (2017), validates that the PINN has learned physically meaningful parameterizations, driven by data and the model's structure.

5.5 Strengths of the Physics-Informed Approach

The successful extraction of parameters in this study, underscores several key strengths of the PINN methodology when applied to well-posed inverse problems with informative data:

- **Intrinsic Regularization by Physics:** The physics loss term itself acts as a powerful implicit regularizer. By compelling the neural network's solution to conform to the underlying true differential equations, it inherently constrains the possible values of the physical parameters to those that permit such physical consistency. This study suggests that when the model is correctly specified with sufficiently informative data, the physics constraint alone can be remarkably effective in guiding parameter extraction towards true values.
- **Data as Primary Driver for Parameter Identification:** When data is generated directly from the true model (even with moderate noise), it contains rich, structured information about the underlying parameters. The PINN's ability to utilise this data effectively, in conjunction with the enforcement of the physical model, is a significant advantage. The accurate extraction of parameters in the absence of explicit parameter regularization implies that the data and physics losses together sculpted a loss landscape with a discernible and attainable minimum at or near the true parameter values.
- **Robustness of the Integrated Framework:** The end-to-end differentiability of the PINN framework, which allows for the simultaneous optimization of neural network weights and the physical parameters using gradient-based methods, is a powerful and flexible feature demonstrated effectively in this work.

5.6 Limitations of the Study and Methodological Considerations

Despite the encouraging results, several limitations and considerations pertinent to this study and the implementation must be acknowledged:

1. **Synthetic Data Context:** The conclusions are drawn from experiments using synthetic data. While this provides a controlled environment for validation against known ground truths, it does not encompass the full spectrum of complexities inherent in real experimental data. Real-world data typically exhibits more intricate noise characteristics, systematic measurement errors, significant device-to-device variability, and cycle-to-cycle stochasticity, all of which could pose greater challenges to the PINN.
2. **Impact of Initial Parameter Guesses:** The comparison between the 2 different runs with one of them having parameters initialised away from the target value than the other

demonstrated that the choice of initial parameter values can influence the final extracted parameters when direct parameter regularization is absent. While Run 1 (with initial `self.a_0p` = 65.61, `self.a_1p` = 4300.0) yielded generally excellent results, run 2 showed that other initializations could lead to better or worse outcomes for specific parameters. This highlights the need for careful consideration of initialization strategies or the use of multiple restarts in practical applications.

3. **Hyperparameter Sensitivity:** The balance between `data_loss_weight` (1000.0) and `physics_loss_weight` (1.0) is critical. The current strong prioritization of data fidelity might render the system more sensitive to noise or model mismatch if applied to experimental data, as the physical constraints would have a relatively weaker influence. The optimal weighting is highly problem-dependent and often requires careful empirical tuning. Similarly, the `dRdt_scale` parameter (1.0×10^4) directly impacts the effective contribution of the physics loss and may need adjustment based on the typical magnitudes of the raw PDE residuals.
4. **Training Duration and Convergence Depth:** The training was conducted for 25,000 epochs. While the loss curves (Figure 4.1) show significant reduction, it is possible that further training, potentially with continued learning rate annealing, could lead to even lower loss values or finer adjustments in some parameters, particularly those that did not achieve perfect target matching (like A_p).
5. **Parameter Identifiability:** While the parameters were accurately recovered in this synthetic test case, this does not constitute a formal proof of unique identifiability for all nine parameters of the Messaris et al. (2017) model solely from the data and physics loss terms. In complex non-linear systems, it is possible for different parameter combinations to yield very similar observable outputs, making their unique disentanglement challenging without strong priors or exceptionally rich and diverse data. The clean convergence observed here suggests that the problem, as configured with this specific synthetic dataset and model structure, was well-posed for these parameters.

5.7 Implications of findings and Broader Context

The successful extraction of the ReRAM model parameters using a PINN framework, particularly when the parameters were initialised close to the target values, carries several important implications and situates this work within a broader context:

1. **Validation of PINNs for Complex Device Model Parameterization:** This study provides further evidence for the utility of PINNs as a sophisticated tool for tackling inverse problems in device physics. The ability to embed the governing differential equations directly into the learning process allows the network to infer parameters that are not only consistent with observed data but also with the underlying physical theory. This is a significant advantage over purely data-driven "black-box" approaches, which may learn to fit data without necessarily capturing the correct physical behaviour or parameter values.
2. **Potential for Automation in Device Characterization:** Traditional parameter extraction for compact models can be a labour-intensive process, often requiring expert

knowledge and iterative manual adjustments. The PINN approach, once the framework is established, offers a pathway towards greater automation. Given a device model (in equation form) and experimental data, a PINN could potentially learn the parameters with less human intervention, accelerating the device characterization workflow.

3. **Robustness to Data Imperfections:** While this study used synthetic data with a controlled noise level, PINNs are theoretically capable of handling noisy and sparse data more effectively than methods that rely solely on data fitting. The physics-based component of the loss function acts as a regularizer, helping to constrain the solution space and prevent overfitting to noise. The successful parameter extraction here, even with a 0.5% noise level, supports this notion. Further investigation with higher noise levels or sparser datasets would be valuable.
4. **Challenges in Hyperparameter Landscape:** The study also implicitly underscores a known challenge with PINNs: the critical importance of hyperparameter tuning. The relative weights of the data loss and physics loss terms, the learning rate, the network architecture, and scaling factors (like `dRdt_scale`) all interact in a complex manner. Finding an optimal set of hyperparameters that ensures rapid convergence to the correct physical parameters across different datasets or model variations remains an active area of research in the PINN community (Wang, Teng and Perdikaris, 2021). The relatively low weight assigned to the physics loss (`physics_loss_weight` = 1.0) compared to the data loss (`data_loss_weight` = 1000.0) in the provided script, for instance, might be suitable for clean, synthetic data but might need re-evaluation for noisy experimental data where stronger enforcement of physics could be beneficial.
5. **Bridging Machine Learning and Physical Sciences:** This project is part of a growing trend of integrating machine learning techniques with domain-specific scientific knowledge. By informing the neural network with the physics of the ReRAM device, the approach utilises the strengths of both data-driven learning and first-principles modelling. This is crucial for advancing scientific discovery and engineering design in complex systems.
6. **Relevance to Next-Generation Computing Hardware:** Accurate and efficient parameter extraction for ReRAM models is vital for the design and simulation of circuits and systems based on these emerging memory technologies. As ReRAM devices are explored for neuromorphic computing, in-memory processing, and advanced memory hierarchies, the ability to rapidly characterize and model them becomes increasingly important. Methodologies like the one explored in this dissertation can contribute to accelerating this development cycle.

The key is the careful formulation of the problem, the accurate encoding of physics, and a methodical approach to training and hyperparameter tuning.

Chapter 6: Conclusion

This project has explored the application of Physics-Informed Neural Networks (PINNs) for the complex task of extracting physical parameters from a TiO_2 Resistive Random-Access Memory (ReRAM) model, specifically the switching rate model proposed by Messaris et al. (2017). The research involved the development of a PINN framework, the generation of synthetic device response data, and the training and evaluation of the model's ability to infer nine key physical parameters. This chapter summarizes the principal findings of the study, reflects on the achievement of the project's aims and objectives, outlines the contributions of this work, and proposes potential directions for future research.

6.1 Summary of Principal Findings

The main findings contain many significant points regarding the efficacy and characteristics of the PINN-based parameter extraction methodology for the target ReRAM model. The PINN framework showcased a strong capability to infer the physical parameters of the ReRAM model with a high degree of accuracy when trained on synthetic data generated with known ground-truth values. In the best run case, the parameters were seen converging very close to the expected target values (e.g., $a_{0,p}$ learned as $71.60345\ \Omega$ vs. target 71.61Ω; $a_{1,p}$ learned as $4368.75854\ \Omega/V$ vs. target $4370.0\ \Omega/V$). This was achieved with particular initial conditions that helped in easy convergence of the parameters. This shows the power of the combined data-fitting and physics-consistency losses to define a landscape conductive to accurate parameter discovery.

A comparative analysis revealed that the choice of initial parameter values can influence the final converged parameters. This points to the non-convex nature of the optimization landscape, where different starting points may lead the optimizer to different local minima. The physics loss component was significantly reduced during training which indicates that the learned solution and the extracted parameters generally adhere well to the governing basic physics differential equation of the ReRAM model.

6.2 Achievements of Aims and Objectives

The TiO_2 ReRAM switching rate model by Messaris et al. (2017) was thoroughly understood and computationally implemented within the PINN framework, with its governing equations forming the core of the physics loss component. The data was generated synthetically for $R(t, V)$ based on the model's analytical solution and known parameters, including appropriate noise simulations defined in `generate_pulse_train_data.`

The PINN architecture was designed and implemented in PyTorch, enabling the Neural Network to approximate $R(t, V)$ while treating the model's physical parameters as trainable variables. The loss function which contains both data loss and physics loss terms was employed which acts as a regularizer for the PINN.

The generated synthetic data was used to train the PINN systematically, leading to the successful learning and extraction of the target physical parameters.

6.3 Contributions of the Project

1. **Demonstration of PINN Efficacy for a Specific ReRAM Model:** It provides a concrete and successful demonstration of applying a PINN framework for the multi-parameter extraction of the Messaris et al. (2017) TiO_2 ReRAM model, highlighting its potential for characterizing such complex non-linear devices.
2. **Insights into PINN Behaviour for Parameter Inference:** This project offers valuable insights into the training dynamics of PINNs for inverse problems, including the influence of initial parameter guess and how the results differed when the loss of each batch were backpropagated to extract parameters. The PINN relies on the information content of the data and the constraining power of the embedded physics.
3. **Methodological Framework and Implementation:** The project presents a complete methodological workflow, from the definition of the physical model and generation of controlled synthetic data to the implementation of the PINN, its training, and comprehensive evaluation.
4. **Highlighting Practical Challenges:** The research also implicitly highlights practical challenges in applying PINNs, such as the sensitivity to hyperparameter tuning (e.g., loss weights, learning rate) and the importance of careful experimental design, particularly concerning data generation and the setup of the learning problem.

6.4 Recommendations for Future Work

This project has showcased the core capabilities of the PINN approach for ReRAM parameter extraction using synthetic data, but there is many possibilities of future research and development to enhance the network's capabilities.

1. **Validation with Real Experimental Data:** The most critical next step is to apply and validate the developed PINN network using real experimental data obtained from physical TiO_2 ReRAM devices. This step will test the performance against actual experimental noise, device-to-device and cycle-to-cycle variability.
2. **Systematic Hyperparameter Optimization and Sensitivity Analysis:** Conduct a comprehensive and systematic optimization study for all key hyperparameters, including the neural network architecture, loss term weights, learning rate strategies and `dRdt_scale` factor. A sensitivity analysis with respect to these hyperparameters would provide a deeper understanding of their impact on parameter accuracy and convergence.
3. **Consistency to Noise and Data:** Investigate the PINN's performance under varying levels of noise and with increasingly sparse datasets to understand and study the limits of robustness and data efficiency.
4. **Addressing Device Variability:** Development of PINN-based approaches that can account device-to-device variability, perhaps by learning distributions of parameters or by using hierarchical Bayesian models.
5. **Extension to Other and More Complex Memristor Models:** Adapt and apply the PINN methodology to extract parameters for other types of memristor models,

including those that may incorporate more complex physical effects (e.g., thermal dynamics, multiple filament types) or possess a larger number of parameters.

6. **Comparative Benchmarking:** Conduct direct comparative studies between the PINN-based extraction method and other established techniques, including the original analytical fitting method proposed by Messaris et al. (2017) and other advanced optimization-based approaches, using identical datasets (both synthetic and experimental).
7. **Transfer Learning and Meta-Learning:** Explore the use of transfer learning (e.g., pre-training on data from one device type and fine-tuning on another) or meta-learning approaches to accelerate parameter extraction for new devices or slightly different models.

By doing these future research directions, the application of Physics-Informed Neural networks can be further refined and validated as a potent and versatile tool for advancing the characterization, modelling and ultimate utilization of ReRAM and other emerging memory technologies.

References

1. *Biolek, Z., Biolek, D. and Biolková, V. (n.d.). SPICE Model of Memristor with Nonlinear Dopant Drift.*
2. *Chen, P.-Y. and Yu, S. (2015). Compact Modeling of RRAM Devices and Its Applications in 1T1R and 1S1R Array Design. IEEE Transactions on Electron Devices, 62(12), pp.4022–4028. doi:https://doi.org/10.1109/ted.2015.2492421.*
3. *Chua, L. (1971). Memristor-The missing circuit element. IEEE Transactions on Circuit Theory, 18(5), pp.507–519. doi:https://doi.org/10.1109/tct.1971.1083337.*
4. *Güneş Baydin, A., Pearlmutter, B., Siskind, J., Baydin, G., Radul, A. and Mark, J. (2018). Automatic Differentiation in Machine Learning: a Survey. Journal of Machine Learning Research, 18, pp.1–43.*
5. *Ielmini, D. (2016). Resistive switching memories based on metal oxides: mechanisms, reliability and scaling. Semiconductor Science and Technology, [online] 31(6), p.063002. doi:https://doi.org/10.1088/0268-1242/31/6/063002.*
6. *Ioannis Messaris, Nikolaidis, S., Serb, A., Stathopoulos, S., Gupta, I., Khiat, A. and Themistoklis Prodromakis (2017). A TiO2 ReRAM parameter extraction method. 2022 IEEE International Symposium on Circuits and Systems (ISCAS), pp.1–4. doi:https://doi.org/10.1109/iscas.2017.8050789.*
7. *Joglekar, Y.N. and Wolf, S.J. (2009). The elusive memristor: properties of basic electrical circuits. European Journal of Physics, 30(4), pp.661–675. doi:https://doi.org/10.1088/0143-0807/30/4/001.*
8. *Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S. and Yang, L. (2021). Physics-informed machine learning. Nature Reviews Physics, [online] 3(6), pp.422–440. doi:https://doi.org/10.1038/s42254-021-00314-5.*
9. *Kvatinsky, S., Ramadan, M., Friedman, E.G. and Kolodny, A. (2015). VTEAM: A General Model for Voltage-Controlled Memristors. IEEE Transactions on Circuits and Systems II: Express Briefs, 62(8), pp.786–790. doi:https://doi.org/10.1109/tcsii.2015.2433536.*
10. *Pan, F., Gao, S.S., Chen, C., Song, C. and Zeng, F. (2014). Recent progress in resistive random access memories: Materials, switching mechanisms, and performance. Materials Science and Engineering R, 83, pp.1–59. doi:https://doi.org/10.1016/j.mser.2014.06.002.*
11. *Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury Google, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., Devito, Z., Raison Nabla, M., Tejani, A., Chilamkurthy, S., Ai, Q., Steiner, B. and Facebook, L. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.*
12. *Prodromakis, T., Toumazou, C. and Chua, L. (2012). Two centuries of memristors. Nature Materials, 11(6), pp.478–481. doi:https://doi.org/10.1038/nmat3338.*
13. *Raissi, M., Perdikaris, P. and Karniadakis, G.E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems*

involving nonlinear partial differential equations. Journal of Computational Physics, 378(378), pp.686–707. doi:https://doi.org/10.1016/j.jcp.2018.10.045.

14. *Serb, A., Bill, J., Khiat, A., Berdan, R., Legenstein, R. and Prodromakis, T. (2016). Unsupervised learning in probabilistic neural networks with multi-state metal-oxide memristive synapses. Nature Communications, [online] 7(1). doi:https://doi.org/10.1038/ncomms12611.*

15. *Strukov, D.B., Snider, G.S., Stewart, D.R. and Williams, R.S. (2008). The missing memristor found. Nature, [online] 453(7191), pp.80–83. doi:https://doi.org/10.1038/nature06932.*

16. *Wang, S., Teng, Y. and Perdikaris, P. (2021). Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. SIAM Journal on Scientific Computing, [online] 43(5), pp.A3055–A3081. doi:https://doi.org/10.1137/20m1318043.*

17. *Waser, R., Dittmann, R., Staikov, G. and Szot, K. (2009). Redox-Based Resistive Switching Memories - Nanoionic Mechanisms, Prospects, and Challenges. Advanced Materials, 21(25-26), pp.2632–2663. doi:https://doi.org/10.1002/adma.200900375.*

18. *Yakopcic, C., Taha, T.M., Subramanyam, G., Pino, R.E. and Rogers, S. (2011). A Memristor Device Model. IEEE Electron Device Letters, 32(10), pp.1436–1438. doi:https://doi.org/10.1109/led.2011.2163292.*

19. *Yu, S. (2017). Neuro-inspired Computing Using Resistive Synaptic Devices. Springer.*

20. *Zahoor, F., Azni Zulkifli, T.Z. and Khanday, F.A. (2020). Resistive Random Access Memory (RRAM): an Overview of Materials, Switching Mechanism, Performance, Multilevel Cell (mlc) Storage, Modeling, and Applications. Nanoscale Research Letters, 15(1). doi:https://doi.org/10.1186/s11671-020-03299-9.*